

PostgreSQL

Parameters Change Runbook for Large Restores

Item	Detail
PostgreSQL Version	16
Replication Type	Streaming Replication (with replication slot)
Data Directory	/apps/pgsql_data/16
Config File	/apps/pgsql_data/16/postgresql.conf

SECTION 1: ALTER SYSTEM vs. postgresql.conf

Recommendation: Use ALTER SYSTEM for temporary tuning changes

Aspect	ALTER SYSTEM	Direct postgresql.conf Edit
How it works	Writes to postgresql.auto.conf (overrides postgresql.conf)	Edits the main config file directly
Risk of error	Low — PostgreSQL validates syntax	High — typos can prevent startup
Audit trail	postgresql.auto.conf shows only overrides	Hard to diff what changed
Rollback	ALTER SYSTEM RESET param; or RESET ALL	Must restore backup file manually
Precedence	postgresql.auto.conf values override postgresql.conf	Base config
Best for	Temporary tuning (like this restore)	Permanent baseline config

Why ALTER SYSTEM Wins for This Scenario

- Your original **postgresql.conf** stays untouched — no backup/restore needed
- Rolling back is a single command: `ALTER SYSTEM RESET ALL;`
- Verify overrides anytime by reading `postgresql.auto.conf`
- No risk of accidentally breaking the config file syntax
- Works identically on both primary and standby

Important Rule: If a parameter exists in BOTH `postgresql.conf` and `postgresql.auto.conf`, the value in `postgresql.auto.conf` ALWAYS wins. Always verify with `SHOW parameter_name;` inside psql.

SECTION 2: Safe Parameter Values for 400 GB Restore

Parameters to Change

Parameter	Default	Restore Value	Why
<code>max_wal_size</code>	1GB	20GB	Reduces checkpoint frequency during heavy writes
<code>min_wal_size</code>	80MB	5GB	Keeps WAL pre-allocated, avoids aggressive recycling
<code>checkpoint_timeout</code>	5min	30min	Longer intervals between checkpoints
<code>checkpoint_completion_target</code>	0.9	0.9	Keep at 0.9 — lower causes faster WAL recycling
<code>wal_buffers</code>	-1 (auto)	256MB	Larger buffer for heavy WAL writes
<code>work_mem</code>	4MB	256MB	Speeds up sorts/joins during FK creation
<code>maintenance_work_mem</code>	64MB	2GB	Speeds up CREATE INDEX, FK validation
<code>wal_keep_size</code>	0	50GB	CRITICAL: Keeps WAL even if standby falls behind
<code>max_slot_wal_keep_size</code>	-1	-1	CRITICAL: Ensure slot is NEVER invalidated
<code>full_page_writes</code>	on	on	NEVER turn off — data corruption risk
<code>synchronous_commit</code>	on	off (opt.)	Tiny crash-risk trade-off for speed

CRITICAL Safety Parameters (Protect Replication)

These two parameters are what prevented your standby from surviving last time:

`wal_keep_size = '50GB'` — Tells PostgreSQL: even after a checkpoint, keep at least 50 GB of old WAL on disk. Safety net if the standby temporarily disconnects.

`max_slot_wal_keep_size = '-1'` — Tells PostgreSQL: never forcibly remove WAL that a replication slot says is still needed. If set to a finite value (e.g. 10GB), PostgreSQL will **INVALIDATE** the slot and remove WAL — which is exactly what happened to you.

SECTION 3: Step-by-Step Procedure

PHASE 1: Pre-Change Verification

Do this FIRST. Do NOT proceed if replication is not healthy.

Run on PRIMARY:

SQL — Check Replication Status

```
-- 1. Check current replication status
SELECT pid, username, client_addr, state, sent_lsn, write_lsn,
       flush_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
```

SQL — Check Replication Slot

```
-- 2. Check replication slot health
SELECT slot_name, slot_type, active, restart_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots;
```

SQL — Record Current Values

```
-- 3. Record current parameter values (save this output)
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW checkpoint_completion_target;
SHOW wal_buffers;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
SHOW synchronous_commit;
```

SQL — Verify Slot Active

```
-- 4. Verify slot is ACTIVE
SELECT slot_name, active FROM pg_replication_slots;
-- Must show: active = t
```

Run on STANDBY:

SQL — Verify Standby Status

```
-- Verify standby is in recovery and streaming
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

PHASE 2: Apply Parameter Changes

Restart Order: STANDBY FIRST, then PRIMARY

The standby is the WAL consumer. It must be up and ready before the primary starts generating heavy WAL from the restore.

Step 2a: Apply on STANDBY

SQL — ALTER SYSTEM on Standby

```
-- Connect to standby via psql and run:
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
```

Restart standby (wal_buffers requires restart):

BASH — Restart Standby

```
# Option A: systemd
sudo systemctl restart postgresql-16

# Option B: pg_ctl
pg_ctl -D /apps/pgsql_data/16 restart
```

Wait and verify standby is streaming again:

SQL — Verify Standby Reconnected

```
-- On STANDBY:
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Wait until: streaming

-- On PRIMARY: confirm slot reconnected
SELECT slot_name, active FROM pg_replication_slots;
-- Must show: active = t BEFORE proceeding
```

DO NOT proceed to Step 2b until the replication slot shows active = t on the primary.

Step 2b: Apply on PRIMARY

SQL — ALTER SYSTEM on Primary

```
-- Connect to primary via psql and run:
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';

-- Optional: for extra speed (tiny crash-safety trade-off)
-- ALTER SYSTEM SET synchronous_commit = 'off';
```

Restart primary:

BASH — Restart Primary

```
# Option A: systemd
sudo systemctl restart postgresql-16

# Option B: pg_ctl
pg_ctl -D /apps/pgsql_data/16 restart
```

Immediately verify replication reconnects:

SQL — Verify Replication After Primary Restart

```
-- On PRIMARY:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots;
-- active must be: t

SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must be: streaming
```

PHASE 3: Perform the 400 GB Restore

Monitor replication every 15-30 minutes during the restore.

SQL — Monitor Slot Lag During Restore

```
-- Run on PRIMARY periodically:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots s
LEFT JOIN pg_stat_replication r ON s.slot_name = r.slot_name;
```

SQL — Check WAL Disk Usage

```
-- Check WAL disk usage:
SELECT pg_size_pretty(sum(size)) AS total_wal_size
FROM pg_ls_waldir();
```

BASH — Check Disk Space

```
# Check disk space from bash:
df -h /apps/pgsql_data/16/pg_wal
```

Warning Signs to Watch For:

- **active = f** on replication slot — standby disconnected, investigate immediately
- **slot_lag growing beyond 30-40 GB** — standby falling behind, but safe if wal_keep_size protects WAL
- **WAL directory consuming too much disk** — may need to pause the restore

PHASE 4: Post-Restore — Revert Parameters

After restore is complete, FK creation is done, and everything is verified.

Step 4a: Revert on STANDBY FIRST

SQL — Reset on Standby

```
-- Option A: Reset ALL overrides at once (recommended)
ALTER SYSTEM RESET ALL;

-- Option B: Reset individual parameters
ALTER SYSTEM RESET max_wal_size;
ALTER SYSTEM RESET min_wal_size;
ALTER SYSTEM RESET checkpoint_timeout;
ALTER SYSTEM RESET checkpoint_completion_target;
ALTER SYSTEM RESET wal_buffers;
ALTER SYSTEM RESET work_mem;
ALTER SYSTEM RESET maintenance_work_mem;
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;
ALTER SYSTEM RESET synchronous_commit;
```

BASH — Restart Standby

```
# Restart standby
sudo systemctl restart postgresql-16
```

SQL — Verify After Standby Revert

```
-- Verify standby is streaming
SELECT pg_is_in_recovery();
SELECT status FROM pg_stat_wal_receiver;

-- On PRIMARY: confirm slot is active
SELECT slot_name, active FROM pg_replication_slots;
```

Step 4b: Revert on PRIMARY

SQL — Reset on Primary

```
ALTER SYSTEM RESET ALL;
```

BASH — Restart Primary

```
# Restart primary
sudo systemctl restart postgresql-16
```

Final Verification:

SQL — Final Verification

```
-- On PRIMARY: confirm parameters are back to original
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW wal_buffers;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;

-- Confirm replication is healthy
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots;

SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
```

Verify postgresql.auto.conf is clean:

BASH — Verify Auto.conf Is Clean

```
cat /apps/pgsql_data/16/postgresql.auto.conf
# After ALTER SYSTEM RESET ALL, this file should be empty
# (or contain only a comment header)
```

SECTION 4: Restart Order Summary

Phase	First	Second
Applying changes (before restore)	STANDBY first — restart, verify streaming	PRIMARY second — restart, verify replication reconnects
Reverting changes (after restore)	STANDBY first — restart, verify streaming	PRIMARY second — restart, verify replication reconnects

The rule is always: STANDBY FIRST, PRIMARY SECOND.

The standby restarts faster. By the time you restart the primary, the standby is already up and ready to consume WAL immediately. This minimizes the window where WAL accumulates with no consumer.

SECTION 5: Emergency — Replication Breaks During Restore

If the standby falls behind and you see:

ERROR: requested WAL segment has already been removed

The only recovery is to rebuild the standby from scratch.

Recovery Steps:

SQL — Drop Broken Replication Slot

```
-- On PRIMARY: drop the broken slot
SELECT pg_drop_replication_slot('your_slot_name');
```

BASH — Rebuild Standby From Scratch

```
# On STANDBY: stop, wipe, rebuild
sudo systemctl stop postgresql-16

# CAREFUL: only do this on the STANDBY!
rm -rf /apps/pgsql_data/16/*

# Rebuild from primary
pg_basebackup -h <primary_host> -p 5432 -U replication_user \
  -D /apps/pgsql_data/16 \
  -Fp -Xs -P -R \
  -S your_slot_name -C

sudo systemctl start postgresql-16
```

Flags explained:

- **-R** — Auto-creates standby.signal and configures primary_conninfo
- **-S slot_name -C** — Creates a new replication slot during the basebackup
- **-Fp** — Plain format (direct file copy)
- **-Xs** — Stream WAL during backup (ensures consistency)
- **-P** — Show progress

SECTION 6: Quick Reference — ALTER SYSTEM Commands

Apply Restore Tuning:

SQL — All ALTER SYSTEM Commands (Copy-Paste Ready)

```
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
```

Revert Everything:

SQL — Single Command Rollback

```
ALTER SYSTEM RESET ALL;
```

Verify Overrides Are Cleared:

BASH — Verify Clean Config

```
cat /apps/pgsql_data/16/postgresql.auto.conf
# Should be empty or only contain a comment header
```

Key Takeaways

- **Always use ALTER SYSTEM** for temporary parameter changes — your postgresql.conf stays untouched
- **Always restart STANDBY first, PRIMARY second** — both when applying and reverting
- **Always set wal_keep_size and max_slot_wal_keep_size = -1** during heavy write operations
- **Always verify replication is healthy** after every restart before proceeding
- **Monitor slot lag during the restore** — catching problems early prevents full standby rebuilds
- **ALTER SYSTEM RESET ALL** is your one-command rollback — no file backups needed